COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	06/10/2026	**DATE SUBMITTED:**	06/10/2026
NAME:	DYLAN PAOLO M MAGTAAN Serrano, Gabriel Immanuel R.	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 11.1 **Basic Algorithm Analysis**

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
      for(int i = 0; i < y.length; i++){
         if(search(x, y[i])!= - 1){
            return false;
         }
      }
   return true;
}
```

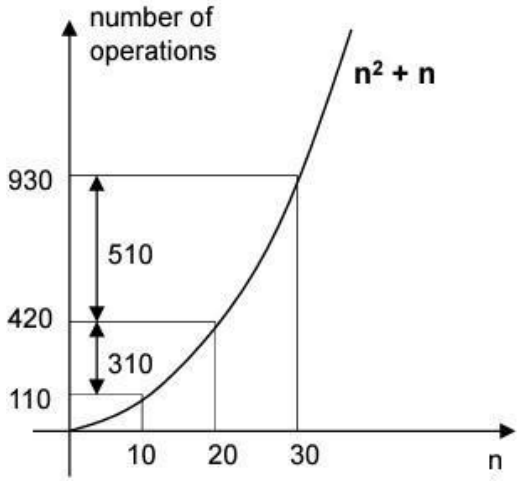
Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: **O(mn)**

- Assume that m = n. (The arrays are the same size)

Provide an analysis of the graph for the worst case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

Analysis:
The following graph illustrates the worst-case scenario for the given code. The outer loop iterates n times, where n is the length of array y. For each iteration, the search() function performs a linear search through array x, which contains m elements. In the worst case, the searched element is not found in x, causing search() to examine all m elements each time. Therefore, the total number of comparisons is approximately mn. Assuming m=n, the total number of comparisons becomes n^2. Thus, the worst-case time complexity is O(n^2).

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Imagine that it is flu season and health department officials are planning to visit a school to ensure that all the enrolled children are administered their flu shot. However, there is a problem: a few children have already taken their flu shots but do not remember if they have been vaccinated against the specific category of flu that the health officials plan to vaccinate all the students against. Official records are sought out and the department is able to find a list of students that have already been administered the vaccine. A small excerpt of the list is shown here:

First Name	Last Name	Flu Shot?
1	3	Yes
2	2	No

Assume that all the names are positive integers and that the given list is sorted. Your task is to write a program that can look up the vaccination status of a given student in the list and outputs to the officials whether the student needs to be vaccinated. Students need to be vaccinated in case of two conditions:

- If they are not present in the list
- If they are present in the list but have not been administered a flu shot

Since the list can have a large number of students, your program should be as fast and efficient as possible. The final output of your program should look as follows:

```
Time taken to search = 45
microseconds Student (836 118)
needs vaccination.
Time taken to search = 37
microseconds Student (836 118)
needs vaccination.
Time taken to search = 53
microseconds Student (836 118)
needs vaccination.
```

The high-level steps for this implementation are as follows:

1. Represent each student as an object of the `Student` class.
2. Overload the required operators for the Student class so that a vector of students can be sorted using the STL `std::sort()` function.
3. Use a **binary search** to see if the student is present on the list.
4. If the student isn't present in the list, your function should return `true` since the student needs to be administered the vaccine.
5. Otherwise, if the student is present in the list but has not been administered the vaccine, return true.
6. Else, return false.

Fill out the table below and include in section 6 table 11-1.

Serrano

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	0 microseconds	Time taken to search = 0microseconds Student (811 915) Needs Vaccination.	Based on this execution speed of 0 microseconds, the search was able to find the student quickly in a small input size
10000	0 microseconds	Time taken to search = 0microseconds Student (576 1326) Needs Vaccination.	Based on this execution speed of 0 microseconds, the search was able to find the student quickly in a slightly larger input size
100000	0 microseconds	Time taken to search = 0microseconds Student (42799 93747) Needs Vaccination.	Based on this execution speed of 0 microseconds, the search was able to find the student quickly in a large input size

Magtaan

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	1 microsecond	Time taken to search = 1microseconds Student (14035 90681) Needs Vaccination.	For an input size of 1000, the program was able to be executed quickly at the speed of 1 microsecond.
10000	1 microsecond	Time taken to search = 1microseconds Student (14035 90681) Needs Vaccination.	At a much larger input size, the program was still able to execute at 1 microsecond.
100000	3 microseconds	Time taken to search = 3microseconds Student (14035 90681) Needs Vaccination.	Even with a 100x increase of the original input size, the execution

			speed increased to 3 microseconds.

Based on our results, the searching algorithm showed that it can execute quickly as the input size increases. Although there is a difference between the two results, it is mostly due to the computer hardware and the system performance during the test.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

For each of the problems below, provide the following:

- Theoretical Analysis
- Experimental Analysis
- Analysis and Comparison

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
for i = 0 to n-1 do
for j = i+1 to n-1 do
if A[i] equals to A[j] then return false
return true
```

Problem 1: Magtaan

Theoretical Analysis
The program uses two nested loops. The outer loop runs from 0 to n-1, while the inner loop compares each element with the elements that come after it, running for n/2 times. So the worst case number of comparison is n(n-1)/2. This results in an O(n^2) time complexity.
Experimental Analysis

```
Problem 1: Unique Elements
Array Size      Execution Time (ns)
100             4800 ns
1000            431500 ns
10000           35587600 ns
```

The results show a significant increase in execution time as the array size grows, which is consistent with the expected O(n^2) time complexity. However, there are variations that may be caused by system conditions and measurement overhead.

Analysis and Comparison

The experimental results agree with the theoretical analysis, as the execution time increases rapidly with larger input sizes. This reflects the O(n^2) time complexity, where the number of comparisons grows quadratically. The experimental results therefore support the theoretical prediction, although the measured times varied due to system conditions.

Problem 1: Serrano

Theoretical Analysis

The algorithm uses two nested for loops to compare each pair of elements in the array. In the worst case, all elements are different, so it checks every possible pair, resulting in O(n^2) time complexity. Its space complexity is 4N + 12 bytes, assuming each array element is an integer that requires 4 bytes and the return value is a boolean that requires 1 byte. In the best case, the first two elements are the same, so the algorithm stops after only 1 comparison, resulting in O(1) time complexity.

Experimental Analysis

```
Array Size      Execution Time (ns)
100             5800 ns
1000            383200 ns
10000           36515600 ns
```

Analysis and Comparison
The experimental result shows that the algorithm took 5,800 ns to process an array of 100 unique elements. As the array size increased to 1,000 and 10,000 elements, the execution time also increased to 383200 ns and 36515600 ns. Based on the results, it agrees with the theoretical analysis because the algorithm compares every pair of the elements when all elements are unique which results in O(n^2) time complexity.

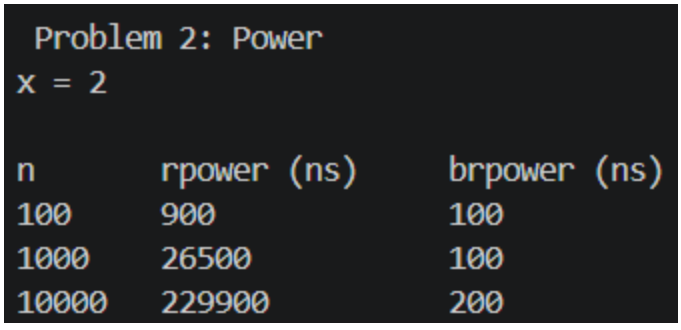
Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

Problem 2: Magtaan

Theoretical Analysis
The rpower function reduces the exponent by 1 in each recursive call, resulting in O(n) time complexity. In comparison, brpower reduces the exponent by approximately half each time, resulting in O(log n) time complexity.
Experimental Analysis



The results show that rpower() takes more time as the input size increases, while brpower() changes very little. This behavior is consistent with their expected O(n) and O(log n) time complexities, although variations occur because of system conditions.

Analysis and Comparison

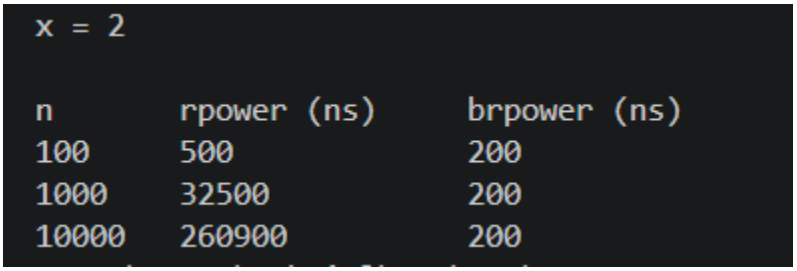
The experimental results support the theoretical analysis. brpower() is more efficient than rpower() for larger inputs because its logarithmic growth requires significantly fewer recursive calls.

Problem 2: Serrano

Theoretical Analysis

The rpower algorithm uses a recursion type that decreases the n by 1 every iteration, this causes the time complexity of O(n). While the brpower algorithm divides the n by 2 in each recursive call, giving us the time complexity of O(log n). We can say that brpower is more efficient than rpower, when we are searching for a large n value.

Experimental Analysis



Analysis and Comparison

The experimental results showed that rpower took 1100 ns for n = 100, 116800 ns for n = 1000, and 276300 ns for n = 10000. While the brpower only took 100 ns, 200 ns, and 200 ns. Based on the results, the experimental analysis is aligned with the theoretical analysis because rpower takes more time as n increases, which follows the O(n) time complexity, while the brpower is much faster as it divides the n value by 2 each recursive call resulting in O(log n) time complexity. We can conclude that brpower is much better when it comes to a larger n value.

3. CONCLUSION

Magtaan

Summary of lessons learned
This activity taught me the difference between theoretical and experimental analysis and how they can be used to interpret system performance. I learned how input size affects execution time and how time and space complexity determine a program's system requirements.
Analysis of the procedure
In ILO A, I learned how to derive the theoretical space and time complexity of a given algorithm and how it can be represented through a graph. In ILO B, I learned how to use both theoretical and experimental analysis to evaluate the efficiency of a program on a system.
Analysis of the supplementary
For the supplementary activities, I enhanced my understanding of theoretical and experimental analysis by applying and comparing them to different programs.
How well did you think you did in this activity?
At first, I had trouble with ILO B because I had only recently encountered the chrono library and were unfamiliar with its functions. However, as I explored its different functions, I gained a better understanding of how to use them and applied them to our programs to compare theoretical and experimental performance.
What are your areas for improvement?
I think I need to become more familiar with the chrono library and the different functions it provides. Aside from that, I could also improve my coding skills by practicing consistently and working on programs regularly.

Serrano

Summary of lessons learned
I learned how to analyze the time complexity and space complexity of algorithms

and understand how the input size affects their performance. I also learned how to compare theoretical analysis with actual execution time and how each computer has a different run time speed based on their hardware.

Analysis of the procedure

The procedure helped me understand how to measure the runtime of an algorithm. In ILO A, I learned how to determine the theoretical time complexity of an algorithm by checking the operations being used. While in ILO B, I learned how to measure the actual execution time by implementing a code that searches randomly generated students and which student does not have a vaccine shot.

Analysis of the supplementary

For the supplementary activity, it helped me apply what I have learned about theoretical and experimental analysis by analyzing different pseudocodes. I learned that theoretical and experimental analysis are closely related because, in Problem 2, the experimental results supported what I analyzed in the theoretical analysis. The brpower algorithm performed much better when analyzing larger values because it has a lower time complexity compared to rpower.

How well did you think you did in this activity?

I think I did well in this activity because I was able to understand the basic concepts and logic of the time complexity and was able to apply them during the procedure part. I was also able to compare the experimental results with the theoretical analysis.

What are your areas for improvement?

I need to improve on analyzing the algorithms and how to properly correlate the theoretical analysis to the experimental analysis. I also need to practice my coding skills on basic algorithms because this topic is new to me and I haven't been able to properly study it.

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.